

Design Patterns

The named, reusable arrangements of objects that professionals reach for by reflex — and the ones that shape every corner of `hathor-core` .

SUBJECT hathor-core · Part I · Track A (programming concepts)

CHAPTER 03 · Foundations · Concepts

BRANCH study/hathor-core-studies

EDITION 2026 · v1

Factory · Builder · Singleton · Adapter · Facade · Proxy · Observer / Publish-Subscribe ·
Creational / Structural / Behavioral

Table of Contents

Chapter 3 — Design Patterns	3
3.1 What a design pattern is (and isn't)	3
3.2 Factory — centralize "which class to create"	4
3.3 Builder — construct a complex object step by step	5
3.4 Singleton — one instance, shared everywhere (handle with care)	5
3.5 Adapter — make an incompatible interface fit	6
3.6 Facade — one simple front over a complex subsystem	7
3.7 Proxy — a stand-in that controls access	7
3.8 Observer / Publish-Subscribe — announce events to interested parties	8
3.9 Bridge — patterns in hathor-core	9
Recap	10

Chapter 3 — Design Patterns

What you'll learn in this chapter

- What a **design pattern** is — a named, reusable solution to a recurring design problem — and what it is not.
- The three families: **creational** (how objects are made), **structural** (how objects are composed), **behavioral** (how objects interact).
- Seven patterns that run through `hathor-core`, each motivated and shown in a small example: **Factory, Builder, Singleton, Adapter, Facade, Proxy, Observer/Publish-Subscribe**.
- The trade-offs — including why one of these (Singleton) is considered a smell as often as a solution.
- A **bridge** mapping each pattern to the exact place it shapes the codebase.

Chapter 1 gave us objects; Chapter 2 gave us time. This chapter is about idioms: the handful of object arrangements that recur so often that engineers gave them names, so they can say "use a factory here" and be instantly understood. Knowing the names is half the battle when reading an unfamiliar codebase — the moment you recognize "ah, this is an Observer," a whole file's intent snaps into focus.

As with the previous two chapters, the body is plain-Python and codebase-agnostic; the closing Bridge (§3.9) maps each pattern onto the real `hathor-core` code you'll meet in Part II.

3.1 What a design pattern is (and isn't)

A **design pattern**¹ is a named, reusable solution to a problem that keeps coming up in software design. It is not code you copy and paste — it is a shape, a way of arranging objects, that you re-implement to fit each situation. The term was popularized in 1994 by a book whose four authors are universally nicknamed the "**Gang of Four**"²; the patterns in this chapter are drawn from that catalogue, chosen because they appear repeatedly in `hathor-core`.

Two things a pattern gives you. First, a **vocabulary**: "wrap it in an adapter," "the manager is a facade," "subscribe an observer" are sentences a team understands without further explanation. Second, a **tested shape**: each pattern is a solution others have already debugged, with known trade-offs, so you are not reinventing it from scratch.

And two cautions. A pattern is a means, not a goal — forcing a pattern where a plain function would do is a classic novice mistake ("pattern-itis"). And every pattern has a cost as well as a benefit; the good engineer reaches for one because its trade-off fits, not because it sounds sophisticated.

The Gang of Four sorted patterns into three families, which is how we'll group them:

CREATIONAL	→ how objects are CREATED	Factory, Builder, Singleton
STRUCTURAL	→ how objects are COMPOSED/wrapped	Adapter, Facade, Proxy
BEHAVIORAL	→ how objects INTERACT/communicate	Observer / Publish–Subscribe

3.2 Factory — centralize "which class to create"

The problem. Code that decides which kind of object to create tends to sprinkle that decision everywhere: `if kind == "circle": ... elif kind == "square": ...`, repeated in a dozen places. When you add a new kind, you must hunt down every one of those spots.

The pattern. A **factory**³ puts the creation decision in one place — a function or method whose job is "given some input, return the right object." Callers ask the factory for an object and stop caring which concrete class they got back (an echo of the abstraction idea from §1.7).

```
def make_shape(kind, **kw):                # the factory
    if kind == "circle":
        return Circle(kw["radius"])
    if kind == "square":
        return Square(kw["side"])
    raise ValueError(f"unknown shape: {kind}")

shape = make_shape("circle", radius=5)    # caller doesn't write Circle(...) itself
```

A common, cleaner variant replaces the `if`-ladder with a **registry** — a dictionary from name to class — so adding a type is one new entry, not a new branch:

```
SHAPES = {"circle": Circle, "square": Square}  # the registry

def make_shape(kind, **kw):
    try:
        return SHAPES[kind](**kw)
    except KeyError:
        raise ValueError(f"unknown shape: {kind}")
```

When to use it. Whenever creation logic is non-trivial or the concrete type depends on data (a string from a config file, a byte tag from the network). The payoff is that the rest of the program is insulated from the `if`-ladder: it asks for a shape and the factory decides the kind.

3.3 Builder — construct a complex object step by step

The problem. Some objects need a lot of parts, many of them optional. Cramming everything into one constructor gives you a monster:

`Pizza(size, crust, cheese, sauce, topping1, topping2, ...)` with a parade of arguments, half of them `None`. This is the telescoping constructor anti-pattern.

The pattern. A **builder**⁴ is a separate object that collects the parts one call at a time and assembles the final object only when you ask. It often returns `self` from each step so calls can be chained — a style called a **fluent interface**⁵:

```
class PizzaBuilder:
    def __init__(self):
        self._toppings = []
        self._size = "medium"

    def size(self, s):
        self._size = s
        return self          # return self → calls can be chained

    def add(self, topping):
        self._toppings.append(topping)
        return self

    def build(self):          # assemble the finished object, once
        return Pizza(self._size, self._toppings)

pizza = PizzaBuilder().size("large").add("cheese").add("mushroom").build()
```

When to use it. When constructing an object takes many steps or many optional pieces, especially when those pieces must be gathered from different places before the object can exist. The builder separates how a thing is assembled from what the thing is — and it is the pattern at the very center of how a node wires itself together (see the Bridge).

3.4 Singleton — one instance, shared everywhere (handle with care)

The problem. Sometimes exactly one of something should exist for the whole program — one configuration object, one connection to a shared resource — and every part of the code should reach the same one.

The pattern. A **singleton**⁶ ensures a class has only one instance and gives everyone a single access point to it. In Python the cleanest form is usually a module-level value, or a function that creates the instance on first call and caches it:

```

_settings = None

def get_settings():
    global _settings
    if _settings is None:
        _settings = Settings(load_from_disk())    # created once, on first call
    return _settings                            # every caller gets the SAME object

```

Why "handle with care." Singleton is the most debated pattern in the catalogue, because it is really **global state**⁷ in a smart coat. Globals make code harder to reason about (any function might secretly depend on or mutate the singleton) and harder to **test** (tests can't easily substitute a fake, and one test's changes leak into the next). Use it only for things that are genuinely single by nature — configuration, the event loop — and prefer passing dependencies in (a technique called dependency injection⁸, which the Builder of §3.3 enables) where you can. We flag this because you will meet singletons in `hathor-core` and should recognize both why they're used and why the codebase limits them.

3.5 Adapter — make an incompatible interface fit

The problem. You have an object that does what you need, but its method names or shape don't match what the surrounding code expects. The code wants `.read()`; the object offers `.fetch()`. You can't (or shouldn't) rewrite either side.

The pattern. An **adapter**⁹ is a thin wrapper¹⁰ that translates one interface into another, presenting the interface the caller expects and forwarding the work to the wrapped object:

```

class LegacySensor:                                # the thing you have (wrong interface)
    def fetch(self):
        return 42

class SensorAdapter:                                # presents the .read() interface the code expects
    def __init__(self, sensor):
        self._sensor = sensor
    def read(self):
        return self._sensor.fetch()                # translate read() → fetch()

reading = SensorAdapter(LegacySensor()).read()      # caller only ever sees .read()

```

When to use it. To bolt together two pieces of code that weren't designed to fit — a third-party library, a legacy class, or two of your own subsystems with mismatched conventions. The adapter localizes the mismatch in one small class, so the rest of the program speaks one consistent interface. It is also how you make two interchangeable backends look identical to their callers — which is exactly how a node hides which event loop it runs on (see the Bridge).

3.6 Facade — one simple front over a complex subsystem

The problem. A subsystem¹² has many moving parts that must be operated in a particular order. Forcing every caller to know that dance — start the CPU, then spin up the disk, then load memory — spreads fragile knowledge everywhere.

The pattern. A facade¹¹ is a single object that offers a simple, high-level interface over a complicated subsystem, doing the intricate coordination behind one or a few easy methods:

```
class Computer:                                # the facade
    def __init__(self):
        self._cpu = CPU()
        self._disk = Disk()
        self._memory = Memory()

    def start(self):                             # one simple call hides the whole sequence
        self._cpu.boot()
        self._disk.spin_up()
        self._memory.load(self._disk.read_boot_sector())
        self._cpu.execute()

Computer().start()                             # callers say "start" and know nothing of the steps
```

When to use it. Whenever a cluster of objects must be coordinated and you want the rest of the program to depend on a simple front rather than the messy internals. The facade doesn't hide the subsystem (advanced callers can still reach the parts) — it provides an easy default path. A node's central coordinator is, at heart, a giant facade (see the Bridge).

3.7 Proxy — a stand-in that controls access

The problem. Sometimes you want to put something between a caller and an object — to delay the object's expensive creation until it's actually needed, to check permissions first, to add logging or caching — without the caller knowing anything changed.

The pattern. A proxy¹³ is a stand-in that implements the same interface as the real object and forwards calls to it, while inserting some control in between. A classic use is **lazy loading**¹⁴ — don't build the costly real object until the first real use:

```

class RealImage:
    def __init__(self, path):
        print(f"loading {path} from disk...") # expensive
        self._path = path
    def display(self):
        print(f"showing {self._path}")

class ImageProxy: # same interface (.display), controls access
    def __init__(self, path):
        self._path = path
        self._real = None
    def display(self):
        if self._real is None: # create the real one only when needed
            self._real = RealImage(self._path)
        self._real.display()

img = ImageProxy("big.png") # nothing loaded yet – cheap
img.display()               # NOW it loads, then displays

```

When to use it. For lazy creation, access control, caching, or remote stand-ins (an object that looks local but forwards to another machine). The proxy and the adapter both wrap — the difference is intent: an adapter changes the interface to fix a mismatch; a proxy keeps the interface and adds control.

3.8 Observer / Publish-Subscribe — announce events to interested parties

The problem. When something notable happens in one part of the program ("a new block was accepted"), several other parts need to react — update an index, notify a websocket client, refresh a metric. Wiring the announcer directly to each reactor (`self.index.update(...); self.websocket.notify(...); self.metrics.bump(...)`) welds them together: the announcer must know about every listener, and adding a listener means editing the announcer.

The pattern. The **observer**¹⁵ pattern — at scale, called **publish-subscribe**¹⁶ — inverts this. The announcer (the subject or publisher) keeps a list of interested parties (observers or subscribers) and notifies all of them when an event occurs, knowing nothing about who they are or what they'll do. Subscribers register a **callback** (Chapter 2!) to be run on each event:


```

class Subject:
    def __init__(self):
        self._subscribers = []

    def subscribe(self, callback):          # listeners register interest
        self._subscribers.append(callback)

    def publish(self, event):              # announce to all, knowing nothing about them
        for callback in self._subscribers:
            callback(event)

bus = Subject()
bus.subscribe(lambda e: print("index updates for", e))
bus.subscribe(lambda e: print("websocket notifies for", e))
bus.publish("new block")
# index updates for new block
# websocket notifies for new block

```

When to use it. Whenever you want to **decouple**¹⁷ event producers from event consumers so each can change independently. Add a new subscriber without touching the publisher; remove one without anyone noticing. This is one of the most consequential patterns in an event-driven system, and it is precisely how a node broadcasts state changes internally and to the outside world. Note how it rests directly on the two prior chapters: it is a list of callbacks (Ch 2) held by an object (Ch 1).

3.9 Bridge — patterns in `hathor-core`

Each pattern above has a home in the codebase. These are forward-pointers; each gets its full treatment in the chapter named.

Bridge — design patterns in the codebase (full treatment in the chapters named):

- **Builder.** The node assembles itself through a builder: `Builder` (tests/simulator) and `CliBuilder` (production) gather storage, indexes, wallet, consensus, and dozens more parts, then produce a wired `HathorManager`. This is the §3.3 pattern at the scale of the whole node — **Chapter 24** (and the `builder` footnote from Ch 1 closes here).
- **Factory.** Twisted builds a fresh protocol object per incoming connection from a `Factory`; the node also uses factories to produce nano-contract runners and to pick a sync agent by negotiated version — §3.2 — **Chapters 34 (protocol factory) & 39 (runner factory)**.
- **Singleton.** The global settings object is fetched through a single accessor (`get_global_settings`) and the reactor is effectively one-per-process — the §3.4 pattern, used sparingly and deliberately — **Chapters 22 (settings) & 16/23 (reactor)**.
- **Adapter.** The reactor wrapper presents one interface over either the Twisted or the asyncio event loop, and the serialization layer uses adapter objects to fit types to the wire format — §3.5 — **Chapters 23 (reactor) & 26 (serialization)**.
- **Facade.** `HathorManager` is a facade: one object exposing `start() / stop()` and high-level operations over storage, consensus, networking, mining, and the wallet — §3.6 at the largest scale — **Chapter 29**.
- **Proxy.** The nano-contract runtime uses proxy/accessor objects to mediate (and control) a contract's access to state; storage scope similarly gates what is reachable — §3.7 — **Chapter 39**.
- **Observer / Publish-Subscribe.** The `PubSubManager` lets internal components subscribe to events (`NEW_BLOCK_ACCEPTED`, ...), and the `EventManager` extends this to a durable, replayable stream for outside consumers — the §3.8 pattern as the node's nervous system, built on Ch 2's callbacks — **Chapter 30**.

Recap

Pattern	Family	One-line intent	Watch for
Factory	Creational	Centralize "which class to create"	the <code>if</code> -ladder or registry dict
Builder	Creational	Assemble a complex object step by step	<code>.build()</code> after chained setup
Singleton	Creational	One shared instance, one access point	hidden global state; test pain

Pattern	Family	One-line intent	Watch for
Adapter	Structural	Translate one interface into another	a wrapper that renames methods
Facade	Structural	One simple front over a complex subsystem	a <code>start()</code> that hides a sequence
Proxy	Structural	Same interface + access control / laziness	a stand-in created before the real one
Observer / Pub-Sub	Behavioral	Notify interested subscribers of events	<code>subscribe()</code> + a list of callbacks

Design patterns are the shared idioms of object-oriented work: named shapes, each a tested answer to a recurring problem, each with a trade-off to respect rather than a badge to collect. You now have the three layers Part II assumes — objects (Ch 1), time and callbacks (Ch 2), and the patterns that arrange them (Ch 3). When Chapter 24 shows a `CliBuilder` gathering parts and calling `build_manager()`, when Chapter 29 presents a `HathorManager` that fronts a dozen subsystems, when Chapter 30 wires a `PubSubManager` full of callbacks — you will not be meeting strangers, only familiar shapes wearing Hathor's names. The remaining concept chapters (wrappers/decorators in Ch 4, typing in Ch 5) round out the programming vocabulary before Track B turns to the blockchain ideas themselves.

1. A design pattern is a named, reusable solution to a commonly occurring problem in software design — a description of a shape to apply, not a finished piece of code. ↩
2. The "Gang of Four" (GoF) are the four authors of *Design Patterns: Elements of Reusable Object-Oriented Software* (1994), the book that catalogued and named 23 classic patterns and made the vocabulary standard. ↩
3. A factory is a function or method whose job is to create and return objects, centralizing the decision of which concrete class to instantiate. Variants include the factory function, factory method, and abstract factory. ↩
4. The builder pattern uses a separate object to construct a complex object step by step, deferring final assembly to a `build()` call. It avoids constructors with long, mostly-optional argument lists. ↩
5. A fluent interface is an API style where methods return the object itself so calls can be chained (`obj.a().b().c()`), reading almost like a sentence. ↩
6. A singleton is a class restricted to a single instance, with one global access point to it. Useful for genuinely-unique resources, but easy to misuse as disguised global state. ↩
7. Global state is data accessible from anywhere in the program. It makes code harder to reason about and test, because any function might read or change it invisibly. ↩
8. Dependency injection is the practice of passing an object its collaborators from outside (e.g. via its constructor) rather than having it create or fetch them itself. It makes dependencies explicit and substitutable (e.g. for testing). ↩

9. An adapter wraps an object to present a different interface than it natively offers, so it can be used where that other interface is expected. Also called a wrapper. ↩
10. A wrapper is any object that encloses another and forwards calls to it, usually adding or altering behavior. Adapters, proxies, and decorators are all kinds of wrapper (decorators are Chapter 4). ↩
11. A facade is an object that provides a simplified, high-level interface to a larger, more complex subsystem, hiding its coordination behind a few easy methods. ↩
12. A subsystem is a cohesive group of classes/modules that together provide some capability (e.g. "storage" or "networking"). A facade fronts a subsystem. ↩
13. A proxy is a stand-in object with the same interface as a real object, forwarding calls to it while inserting control such as lazy creation, access checks, caching, or remote communication. ↩
14. Lazy loading (lazy initialization) means deferring the creation of an expensive object until the first moment it is actually needed, rather than up front. ↩
15. The observer pattern lets an object (the subject) maintain a list of dependents (observers) and notify them automatically when its state changes, typically by calling a registered callback. ↩
16. Publish-subscribe is the observer pattern at system scale: publishers emit events to a channel/bus without knowing the subscribers, and subscribers receive events without knowing the publishers. Maximum decoupling. ↩
17. Decoupling means reducing how much two parts of a system need to know about each other, so each can change independently without breaking the other. ↩